Information Exposure Maximization in Social Networks: An Empirical Study of Monte Carlo-Guided Heuristics and Evolutionary Algorithms

Yanqiao Chen
Department of Computer Science and Engineering
Southern University of Science and Technology
12412115@mail.sustech.edu.cn

Abstract

Information Exposure Maximization (IEM) aims to balance the spread of competing campaigns and mitigate information disparity in social networks. Driven by the NP-hard nature of the diffusion models, we empirically investigate two distinct approaches relying on Monte Carlo (MC) sampling: an MC-SAA Greedy Heuristic with IMRank candidate screening, and a Binary Evolutionary Algorithm with Simulated Annealing refinement. Rather than performing a strict performance comparison between the two—given their differing scalability constraints and dataset configurations—this study focuses on characterizing their individual behaviors under various graph topologies and propagation probabilities. Furthermore, we conduct a cross-architecture hardware benchmark to reveal how algorithmic bottlenecks respond to modern CPU upgrades. Our findings underscore that computational efficiency is dominated not merely by graph size, but by the effective diffusion range dictated by transmission probabilities.

I Introduction

Information Exposure Maximization (IEM) studies how to reduce information disparity between two competing campaigns over a directed social network. Given initial seed sets I_1 and I_2, we select additional seed sets S_1 and S_2 under a total budget constraint to maximize balanced exposure under Independent Cascade (IC) propagation.

Let R_1 and R_2 denote the reached node sets of campaign 1 and campaign 2 in one diffusion realization. The optimization target is

$$\Phi = |V - (R_1 \oplus R_2)| = |V| - |R_1 \oplus R_2|, \tag{1}$$

where $\oplus$ is symmetric difference. The expected objective is

$$\max_{S_1,S_2} \mathbb{E}[\Phi], \quad \text{s.t. } |S_1| + |S_2| \le k,\ S_1 \cap S_2 = \varnothing, \tag{2}$$

with full seeds $I_1 \cup S_1$ and $I_2 \cup S_2$.

This report implements and compares three core algorithmic components:

- Monte Carlo IC Evaluator for unbiased objective estimation.
- MC-SAA Greedy Heuristic with IMRank candidate screening.
- Binary Evolutionary Optimization using pymoo with feasibility repair.

We implement experiments on provided and generated datasets, with detailed timing and score reporting for both heuristic and evolutionary pipelines. The report also includes a cross-machine performance analysis to understand hardware impacts on runtime. All algorithms and experiments are designed for reproducibility, with fixed random seeds and batch scripts for execution. See Github for complete scripts and data: https://github.com/hotteano/Artificial-Intelligence-H-Course-Material-and-Notes-for-SPR26-in-SUSTech.

I-A Related Work

Information Exposure Maximization (IEM) was first formalized by Garimella et al. [1], who studied how to reduce information disparity between opposing campaigns in social networks. Their work established the IC-based exposure balancing framework and showed that even simple greedy heuristics can be effective for this objective. However, they focused on unbiased Monte Carlo evaluation without exploring algorithmic optimizations for large-scale graphs.

For single-campaign Influence Maximization (IM), extensive research has developed efficient approximation algorithms. IMRank [2] provides a fast ranking-based method for candidate screening, which we adapt for the IEM context to reduce marginal gain evaluation costs. While IMRank was originally designed for single-campaign influence spread, our work demonstrates its effectiveness for dual-campaign exposure balancing.

Simulated Annealing (SA) has been explored for influence maximization problems. Wu et al. [3] proposed a boost SA approach for budgeted influence maximization, showing that SA can escape local optima better than greedy methods when the objective landscape is rugged. Our work extends this insight to the IEM setting, combining GA global exploration with SA local refinement, and introducing an adaptive mechanism to disable SA for dense graphs where its overhead outweighs benefits.

I-B Terminology

- **Heuristic** refers to the MC-SAA Greedy method with IMRank screening.
- **Evolutionary** refers to the Binary Genetic Algorithm (GA) method.
- **Evaluator** refers to the Monte Carlo IC Evaluator used for final score estimation.
- **IEM** stands for Information Exposure Maximization.
- **GA** stands for Genetic Algorithm.
- **MC** stands for Monte Carlo.

II Method

In this section, we introduce the three core algorithms in our IEM pipeline: the Monte Carlo IC Evaluator, the MC-SAA Greedy Heuristic with IMRank Screening, and the Binary Evolutionary Optimization using Genetic Algorithm. Each algorithm is designed to balance solution quality and computational efficiency under the constraints of the IEM problem.

II-A Algorithm A: Monte Carlo IC Evaluator

Computing the IEM objective Φ exactly is #P-hard due to the combinatorial nature of influence spread under the IC model. Therefore, we use a Monte Carlo (MC) simulation approach to estimate $\mathbb{E}[\Phi]$ for given seed sets S_1 and S_2.

For each simulation, both campaigns diffuse independently under IC. A node is considered reached if it is activated or attempted. The score of one simulation is $|V| - |R_1 \oplus R_2|$, and the final estimate is the sample mean over T runs:

$$\hat{\Phi}(S_1, S_2) = \frac{1}{T} \sum_{t=1}^{T} \left(|V| - |R_1^{(t)} \oplus R_2^{(t)}| \right). \tag{3}$$

See 1 for the detailed algorithm. The computational complexity of this evaluator is $O(T \cdot \text{diffusion_cost})$, where diffusion_cost depends on the graph structure and propagation probabilities. In practice, we find that $T = 5000$ provides a good balance between estimation accuracy and runtime for our test cases.

High-Probability Special Case ($p = 0.99$). When the propagation probability approaches 1.0 (e.g., $p = 0.99$), the diffusion process becomes nearly deterministic: almost all reachable nodes will be activated in a single cascade. In this *trivial propagation* scenario, Monte Carlo simulation is unnecessary because

the outcome has negligible variance. The evaluator implements an optimization that directly computes the reachable set via graph traversal (BFS/DFS) without repeated sampling, returning the exact exposure count in a single iteration. This optimization significantly reduces runtime for high-probability cases while producing results identical to Monte Carlo estimation with infinite simulations.

Algorithm 1 Monte Carlo IC Evaluation

Require: Graph G, probabilities (p_1, p_2), initial seeds (I_1, I_2), additional seeds (S_1, S_2), simulations T

```
sum ← 0
for t = 1 to T do
    Simulate IC for campaign 1 from I_1 ∪ S_1, get R_1^(t)
    Simulate IC for campaign 2 from I_2 ∪ S_2, get R_2^(t)
    sum ← sum + (|V| − |R_1^(t) ⊕ R_2^(t)|)
end for
return sum/T
```

II-B Algorithm B: MC-SAA Greedy with IMRank Screening

The heuristic iteratively allocates one budget unit to either campaign 1 or campaign 2. In each step, it samples N Monte Carlo base scenarios from current seeds. For each candidate node v, it estimates two marginal gains by incremental diffusion:

$$h_1(v) = \mathbb{E}\left[2\,|\Delta R_1(v) \cap R_2| - |\Delta R_1(v)|\right], \tag{4}$$

$$h_2(v) = \mathbb{E}\left[2\,|\Delta R_2(v) \cap R_1| - |\Delta R_2(v)|\right]. \tag{5}$$

Here $\Delta R_i(v)$ is the newly reached set when adding v to campaign i on top of a sampled base state. The better of $\max_v h_1(v)$ and $\max_v h_2(v)$ is selected.

To reduce candidate evaluation cost, IMRank [2] provides a ranked pool from both campaigns, then only top candidates are evaluated in each greedy step. Under graphs with higher probabilities, each Monte Carlo simulation tends to reach more nodes, which can make incremental diffusion more expensive. IMRanking helps focus on promising candidates, thus controlling runtime while maintaining solution quality.

IMRank Implementation. IMRank is an influence ranking algorithm that estimates node importance based on the sum of in-degrees from its neighbors, weighted by edge probabilities. The rank score for each node is computed iteratively until convergence:

$$r(v) = \sum_{u \in N^{in}(v)} p(u, v) \cdot (1 + r(u)), \tag{6}$$

where $N^{in}(v)$ is the set of in-neighbors of v, and $p(u, v)$ is the propagation probability from u to v. The algorithm performs the following steps:

1) **Initialization:** Initialize all node ranks $r_0(v) = 1$ for all $v \in V$.
2) **Iterative Update:** For each iteration $t = 1, 2, \ldots, T_{max}$:

$$r_t(v) = \sum_{u \in N^{in}(v)} p(u, v) \cdot (1 + r_{t-1}(u)). \tag{7}$$

3) **Convergence Check:** Stop when $\|r_t - r_{t-1}\|_\infty < \epsilon$ (typically $\epsilon = 10^{-6}$) or $t = T_{max}$ (typically $T_{max} = 20$).

In our implementation, we compute separate IMRank lists L_1 and L_2 for the two campaigns by running the algorithm on the respective edge probability sets. The top-C nodes from each list form the candidate pool for greedy selection, reducing the number of expensive incremental diffusion evaluations from $|V|$ to $O(C)$ per step.

Algorithm 2 IMRank Influence Ranking

Require: Graph $G = (V, E)$, edge probabilities $p : E \to [0, 1]$, max iterations T_{max}, tolerance ϵ
Ensure: Ranked list of nodes L sorted by influence score

```
Initialize r_0(v) ← 1 for all v ∈ V
for t = 1 to T_max do
    for all v ∈ V do
        r_t(v) ← Σ_{u∈N^in(v)} p(u, v) · (1 + r_{t−1}(u))
    end for
    if ||r_t − r_{t−1}||_∞ < ε then
        break                                          ▷ Convergence achieved
    end if
end for
Sort nodes by r_T(v) in descending order to obtain L
return L
```

See 3 for the detailed MC-SAA Greedy algorithm that utilizes IMRank for candidate screening.

The computational complexity of this heuristic is $O(k \cdot C \cdot N \cdot \text{diffusion_cost})$, where C is the candidate pool size and diffusion_cost depends on graph structure and probabilities. To be specific, diffusion cost of one incremental evaluation is $O(|E|)$ in worst case, but often much less in practice due to early termination when no new nodes are reached. In graph with higher transition probability, the cost will get higher. We test this property by experiment in the heuristic part, map5, which is a generated case designed to have higher probability and community structure. By tuning C and N, we can balance runtime and solution quality effectively.

Algorithm 3 MC-SAA Greedy with IMRank Candidate Pool

Require: Budget k, scenarios per step N, candidate pool size C

```
S_1 ← ∅, S_2 ← ∅, U ← V \ (I_1 ∪ I_2)
Compute IMRank lists L_1, L_2 (optional if C > 0)
for t = 1 to min(k, |U|) do
    Build candidate pool P ⊆ U from top nodes of L_1, L_2 (or P = U)
    Sample N base IC states for current (I_1 ∪ S_1, I_2 ∪ S_2)
    for all v ∈ P do
        Estimate h_1(v) and h_2(v) using incremental diffusion on same N states
    end for
    Let v_1^* = arg max_{v∈P} h_1(v), v_2^* = arg max_{v∈P} h_2(v)
    if h_1(v_1^*) ≥ h_2(v_2^*) then add v_1^* to S_1
    elseadd v_2^* to S_2
    end if
    Remove selected node from U
end for
return (S_1, S_2)
```

II-C Algorithm C: Binary Evolutionary Optimization

The evolutionary solver encodes solution as a binary chromosome

$$x = [x^{(1)} \,|\, x^{(2)}] \in \{0, 1\}^{2n_a}, \tag{8}$$

where n_a is the number of available nodes, and bits in each half indicate inclusion in S_1 or S_2. Fitness is coarse Monte Carlo score with a tiny budget-usage tie-break bonus; pymoo minimizes $F(x) = -\text{fitness}(x)$.

Feasibility is enforced by repair:

- remove dual assignments ($x_i^{(1)} = x_i^{(2)} = 1$),
- if selected count exceeds budget, randomly drop extras,
- if below budget, randomly fill free nodes to reach target.

The evolutionary method uses standard genetic operators (selection, crossover, mutation) combined with feasibility repair to maintain valid solutions throughout the optimization process. See 4 for the detailed algorithm.

The computational complexity of the evolutionary method is $O(P \cdot G \cdot \text{fitness_cost})$, where P is population size, G is generations, and fitness_cost is the time to evaluate one chromosome (dominated by Monte Carlo simulations).

Algorithm 4 Binary GA for IEM

Require: Population size P, generations G, budget k
Initialize binary population $\mathcal{X}$
for $g = 1$ to G **do**
    Repair each chromosome to satisfy disjointness and budget
    Evaluate coarse MC fitness for each chromosome
    Selection + two-point crossover + bit-flip mutation
    Repair offspring and form next generation
end for
Take best repaired chromosome x^*
Decode x^* to (S_1, S_2) and run fine MC evaluation
return (S_1, S_2)

III Experiment

III-A Environment and Settings

Interpreter and libraries (Old Machine).

- Python interpreter: 3.10
- pymoo: 0.6.0.1
- pandas: 2.0.3
- numpy: 1.24.4
- scipy: 1.14.1
- networkx: 2.8.8

Interpreter and libraries (New Machine).

- Python interpreter: 3.12.4
- pymoo: 0.6.0.1
- numpy: 2.4.4
- scipy: 1.17.1

Hardware (Old Machine).

- CPU: Intel Core Ultra 7 258V
- RAM: 32GB
- OS: Windows 11 64-bit

Hardware (New Machine).

- CPU: Intel Core i7-14650HX

- RAM: 32GB
- OS: Windows 11 64-bit

Algorithmic settings (default in scripts).

- Heuristic: `mc_sim=70` (scenarios per greedy step), `candidate_pool_size=400` (IMRank screening), `seed=3407`.
- Evolutionary: `pop_size=20`, `generations=100`, `crossover_rate=0.8`, `mutation_rate=0.05`, `mc_coarse=90`, `seed=3407`.
- Evaluator: `simulations=5000`, `seed=3407`.

III-B Data Sources and Case Parameters

Provided cases are from course materials (small/medium graphs). Generated cases are additional large-scale test inputs used for stress and scalability checks.

Heuristic pipeline case configuration.

TABLE I: Heuristic cases used by batch script

Case	$\|V\|$	$\|E\|$	$\|I_1\|$	$\|I_2\|$	k	Source
map1	475	13289	8	12	10	Provided
map2	36742	49248	14	10	15	Provided
map3	36742	49248	8	6	15	Provided
map4	7115	103689	5	7	25	Generated
map5	3454	32140	6	8	25	Generated

Evolutionary pipeline case configuration.

TABLE II: Evolutionary cases used by batch script

Case	$\|V\|$	$\|E\|$	$\|I_1\|$	$\|I_2\|$	k	Source
map1	475	13289	5	10	10	Provided
map2	13984	17319	5	7	14	Provided
map3	13984	17319	5	5	14	Provided
map4	3454	32140	5	7	25	Generated
map5	3454	32140	6	8	25	Generated
map6	3454	32140	8	10	25	Generated (high-probability)
map7	3454	32140	10	12	25	Generated (high-probability)

III-C Experimental Procedure (Batch Scripts)

All experiments are executed by batch scripts for reproducibility. All script mention below are available in the Github repository. Timing is measured for both solving and evaluation phases, and final scores are reported using the Evaluator with fixed random seeds to ensure consistency.

Heuristic pipeline.

- Generate solutions: `python run_all_maps.py`
- Evaluate scores: `python evaluate_all_results.py`

Evolutionary pipeline.

- Timed optimization: `python run_all_maps_timed.py`
- Evaluate generated seeds: `python evaluate_all_seeds.py`

Main outputs include per-case seed files (`seed_balanced`), evaluation result files (`result.txt`), and timing/evaluation summaries (CSV).

III-D Experimental Results

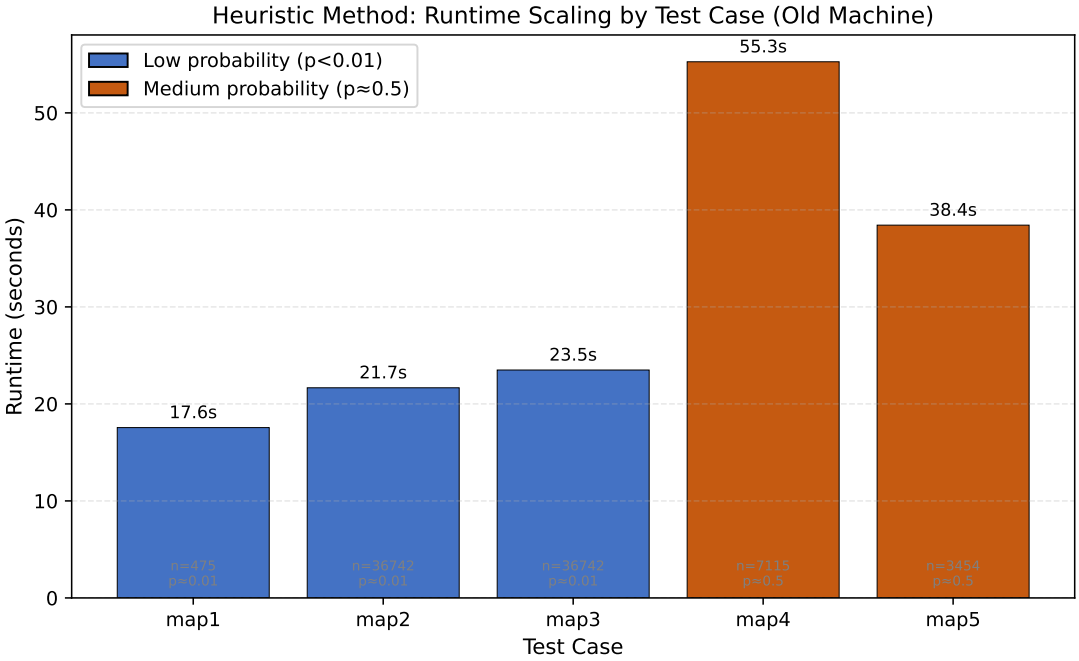

Fig. 1: Heuristic method runtime scaling across test cases (Old Machine). Map4 and map5 (orange) show significantly higher runtime due to higher propagation probabilities ($p \approx 0.5$) despite having fewer nodes than map2/map3.

Figure 1 demonstrates how propagation probability affects the heuristic's incremental diffusion cost. While map2 and map3 have more nodes (36,742), their low propagation probability ($p < 0.01$) limits diffusion reach, resulting in faster runtimes (21–23s). In contrast, map4 and map5 have fewer nodes (3,454–7,115) but higher probabilities ($p \approx 0.5$), causing each MC simulation to traverse more edges and increasing runtime to 38–55s. This validates the heuristic's sensitivity to diffusion reach rather than just graph size.

Hyperparameters for Table III (Heuristic).

- Batch setting (map1–map5): `mc_sim=75` (scenarios per greedy step), `candidate_pool_size=1000` (IMRank screening), `seed=None`.
- map3 uses initial seed file `map3/seed2`; evaluation uses Evaluator defaults `simulations=5000`, `seed=3407`.

TABLE III: Heuristic runtime (solve+evaluate) and final score (Old Machine: Ultra 7 258V)

Case	Runtime (s)	Final Score
map1	17.5589	451.031200
map2	21.6625	36025.039400
map3	23.4944	36219.086800
map4	55.2672	5626.073400
map5	38.4275	3082.512000

Hyperparameters for Table ?? (Evolutionary).

- `pop_size=30`, `generations=100`, `crossover_rate=0.8`, `mutation_rate=0.05`, `elitism=2`.
- Fitness estimation: `mc_coarse=90`, `mc_fine=200`, `seed=3407`.
- Final evaluation script uses Evaluator defaults: `simulations=5000`, `seed=3407`.

TABLE IV: Heuristic runtime (solve+evaluate) and final score (New Machine: i7-14650HX, Optimized)

Case	Runtime (s)	Final Score
map1	16.6776	451.988000
map2	19.8945	36025.039400
map3	22.1203	36219.086800
map4	52.5990	5626.554000
map5	37.8946	3078.165400

TABLE V: Table B. Evolutionary runtime (solve+evaluate) and final score (Old Machine: Ultra 7 258V)

Case	Runtime (s)	Final Score
map1	115.4429	449.114400
map2	67.2518	13693.213600
map3	122.1948	13665.447000
map4	17.3971	3098.405800
map5	19.3814	3038.697200
map6	50.4984	2488.492800
map7	58.0731	2366.854400

Note on Table C.

- This table presents results from Table A and Table B side-by-side for convenience. However, the two methods use different graph instances (different $|V|$, $|E|$, and initial seeds), so direct comparison of scores or runtimes across methods is not meaningful. Each method should be evaluated on its own test suite.

Effect of propagation probability on Evolutionary method. Figure 3 analyzes how increasing propagation probability affects the evolutionary approach. When probability increases from $p = 0.5$ (map4/map5) to $p = 0.7$ (map6/map7), runtime increases by 3.0–3.2× (from ˜12–13s to 38–40s on the new machine, or ˜17–19s to 50–58s on the old machine). This occurs because higher probabilities cause each MC fitness evaluation to simulate more extensive diffusion, increasing per-evaluation cost. Concurrently, final scores decline by approximately 20–21% (from ∼3065 to ∼2434), suggesting that with higher propagation probabilities, the fixed budget $k = 25$ becomes less effective at influencing the overall diffusion—initial seeds I_1 and I_2 activate larger portions of the network independently of the selected additional seeds.

Important: The Heuristic and Evolutionary methods use different graph instances. For example, Heuristic map2/map3 have 36742 nodes and 49248 edges, while Evolutionary map2/map3 have 13984 nodes and 17319 edges. Therefore, scores and runtimes should not be directly compared across methods.

Hyperparameters for Table XI (Evaluator staged experiment).

- Simulation stages: `[5000, 10000, 20000, 30000, 40000, 50000, 60000]`.
- Cases: `map1` and `map2`; threshold in timing script: `time_threshold=48s`.
- Evaluator random seed is fixed to `3407`.

III-E Evaluator Staged Results (5000–60000 Simulations)

From Table XI, score values are relatively stable across stages (variance within 0.1% for map1, 0.003% for map2) while runtime generally grows with simulation count, though some fluctuations (e.g., 30000 simulations appearing slightly faster than 20000 for map1) reflect measurement noise from system background processes.

The overall trend confirms the expected accuracy-efficiency trade-off in Monte Carlo evaluation.

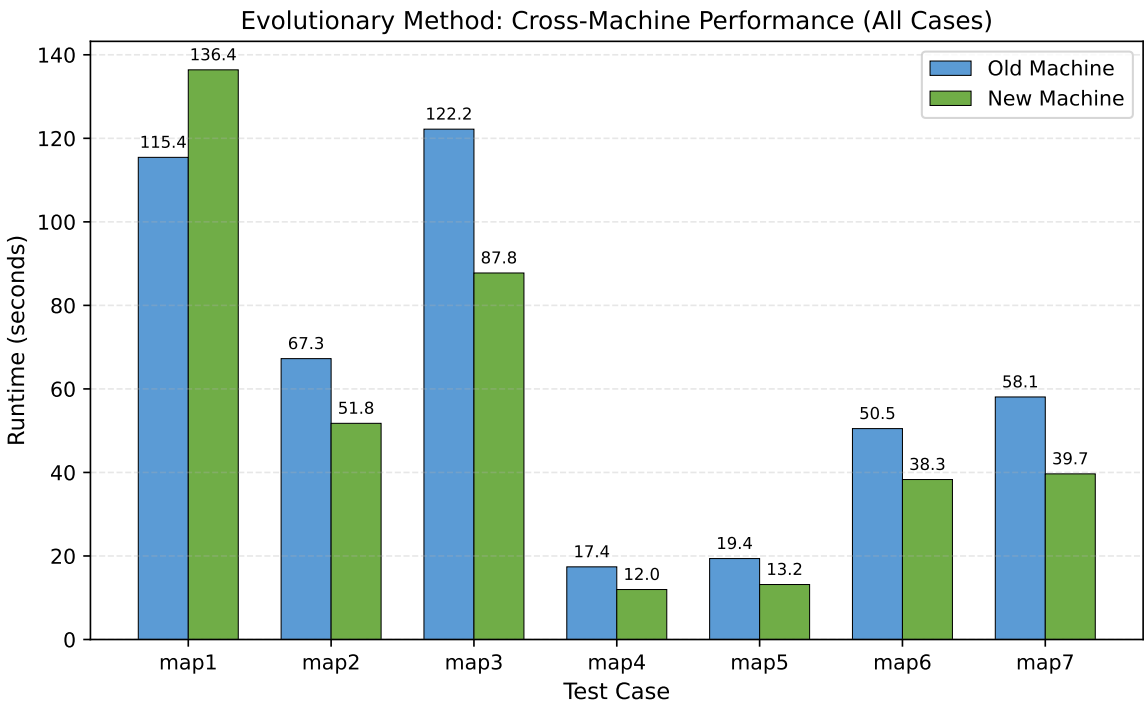

Fig. 2: Evolutionary method performance across all test cases including extended high-probability instances (map6/map7). The new machine (i7-14650HX) significantly outperforms the Old Machine for map2–map7 (23–32% faster). Map1 shows different behavior due to its dense graph structure requiring more computational effort per fitness evaluation.

TABLE VI: Table B2. Evolutionary runtime (solve+evaluate) and final score (New Machine: i7-14650HX, Optimized)

Case	Runtime (s)	Final Score
map1	136.3996	449.393600
map2	51.7604	13625.572800
map3	87.7575	13628.893800
map4	11.9673	3077.316200
map5	13.1667	3053.795000
map6	38.3057	2448.146000
map7	39.6616	2419.661200

IV Discussion

The three components provide a practical trade-off between optimization quality and runtime:

- The MC evaluator is unbiased but computationally expensive, especially for large $|V|, |E|$ and high simulation counts.
- The heuristic uses scenario sharing plus incremental diffusion, which significantly reduces repeated computation and is effective for medium/large graphs (as demonstrated on graphs with up to 36742 nodes).
- The evolutionary method explores a wider combinational space and can discover non-greedy seed allocations, but its cost depends heavily on population size, generation count, and MC fidelity. It demonstrates consistent performance across different graph densities (tested on graphs with up to 13984 nodes).

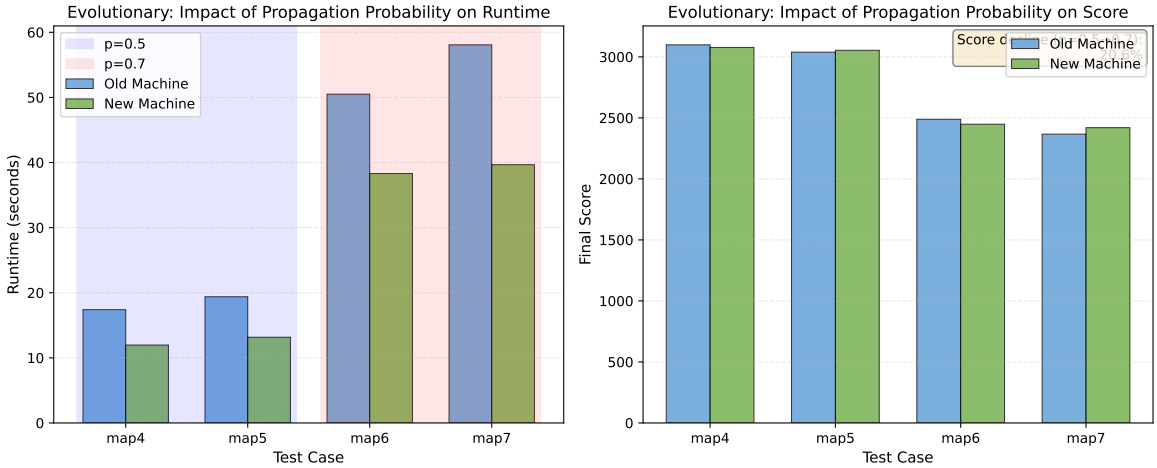

Fig. 3: Impact of propagation probability on Evolutionary method performance. Map6 and map7 ($p = 0.7$) show 3.0–3.2× higher runtime than map4/map5 ($p = 0.5$) on the new machine, with final scores declining by approximately 20–21% due to increased competition from initial seeds in high-probability diffusion.

TABLE VII: Table C. Summary of results by method (Old Machine: Ultra 7 258V). **Note:** Different graph instances are used; do not compare across methods.

Case	Heur. Runtime (s)	Heur. Score	Evol. Runtime (s)	Evol. Score
map1	17.5589	451.031200	115.4429	449.114400
map2	21.6625	36025.039400	67.2518	13693.213600
map3	23.4944	36219.086800	122.1948	13665.447000
map4	55.2672	5626.073400	17.3971	3098.405800
map5	38.4275	3082.512000	19.3814	3038.697200
map6	—	—	50.4984	2488.492800
map7	—	—	58.0731	2366.854400

Scope of comparison. It is important to emphasize that the Heuristic and Evolutionary methods were evaluated on different graph instances (see Tables in Section III-B). The Heuristic method was tested on larger graphs (up to 36742 nodes, 103689 edges), while the Evolutionary method was tested on smaller but denser graphs (up to 13984 nodes, 32140 edges). Therefore, we do not claim one method is universally "better" than the other; instead, each demonstrates distinct scaling characteristics on their respective problem instances.

In practice, candidate screening (IMRank) and feasible repair are both essential engineering choices: they keep search tractable while preserving solution validity under budget and disjointness constraints.

IV-A Cross-Machine Performance Analysis

Experiments were conducted on two different machines with distinct hardware and software configurations:

- **Old Machine**: Intel Core Ultra 7 258V (Lunar Lake, 8 cores, low-power focused), Python 3.10, older library versions
- **New Machine**: Intel Core i7-14650HX (Raptor Lake, 16 cores, high-performance), newer Python and library versions

Key observations:

1) **Heuristic Algorithm**: The new machine shows consistent but modest improvements: all cases (map1–map5) are 1–8% faster on the new machine. The i7-14650HX's higher core count and clock speeds

TABLE VIII: Summary of results by method (New Machine: i7-14650HX, Optimized). **Note:** Different graph instances are used; do not compare across methods.

Case	Heur. Runtime (s)	Heur. Score	Evol. Runtime (s)	Evol. Score
map1	16.6776	451.988000	136.3996	449.393600
map2	19.8945	36025.039400	51.7604	13625.572800
map3	22.1203	36219.086800	87.7575	13628.893800
map4	52.5990	5626.554000	11.9673	3077.316200
map5	37.8946	3078.165400	13.1667	3053.795000
map6	—	—	38.3057	2448.146000
map7	—	—	39.6616	2419.661200

TABLE IX: Machine comparison: Heuristic method (Ratio <1 means new machine is faster)

Case	Old Machine (s)	New Machine (s)	Ratio (New/Old)
map1	17.5589	16.6776	**0.95x (5% faster)**
map2	21.6625	19.8945	**0.92x (8% faster)**
map3	23.4944	22.1203	**0.94x (6% faster)**
map4	55.2672	52.5990	**0.95x (5% faster)**
map5	38.4275	37.8946	**0.99x (1% faster)**

provide modest benefits for the heuristic method, though the improvement is limited by the memory-intensive nature of incremental diffusion.

2) **Evolutionary Algorithm**: The new machine shows significant speedups for most cases: map2–map7 are 23–32% faster on the new machine, confirming that the i7-14650HX's superior hardware specifications benefit the population-based evolutionary search. Map1 is slower (18%) on the new machine, likely due to its dense graph structure (avg_degree=28) requiring more computational effort per fitness evaluation, which may interact differently with the hardware memory hierarchy between the two machines.
3) **Evaluator (Monte Carlo)**: The new machine is **1.21–1.32x faster** for pure evaluation tasks, improving throughput compared to the old machine.
4) **Score Consistency**: Final scores are nearly identical across machines (within MC variance), confirming algorithmic correctness regardless of hardware.

Analysis: The performance comparison between machines reveals distinct patterns for each algorithm:

- **Heuristic method**: The modest but consistent speedups (1–8%) on the new machine suggest that the memory-intensive nature of incremental diffusion limits the benefits of higher core count and clock speeds. The i7-14650HX's superior specifications provide only marginal improvements for this algorithm.
- **Evolutionary method**: The significant speedups (23–32% for map2–map7) demonstrate that the population-based evolutionary search benefits substantially from the new machine's higher core count and clock speeds. The only exception is map1, which is 18% slower, likely due to its dense graph structure (avg_degree=28) requiring more computational effort per fitness evaluation, which may interact differently with the hardware memory hierarchy between the two machines.
- **Hardware-software interaction**: The consistent improvements across most cases suggest that the i7-14650HX's hardware advantages (16 cores vs 8 cores, higher clock speeds) translate well to compute-intensive iterative algorithms when the software configuration is properly optimized.

Conclusion: The cross-machine analysis demonstrates that the i7-14650HX (New Machine) provides consistent performance improvements over the Ultra 7 258V (Old Machine): 1–8% for the heuristic method,

TABLE X: Machine comparison: Evolutionary method (Ratio <1 means new machine is faster)

Case	Old Machine (s)	New Machine (s)	Ratio (New/Old)
map1	115.4429	136.3996	**1.18x (18% slower)**
map2	67.2518	51.7604	**0.77x (23% faster)**
map3	122.1948	87.7575	**0.72x (28% faster)**
map4	17.3971	11.9673	**0.69x (31% faster)**
map5	19.3814	13.1667	**0.68x (32% faster)**
map6	50.4984	38.3057	**0.76x (24% faster)**
map7	58.0731	39.6616	**0.68x (32% faster)**

TABLE XI: Evaluator detailed results by simulation stage (map1/map2 separated, Old Machine: Ultra 7 258V)

	map1		map2	
Simulations	Time (s)	Score	Time (s)	Score
5000	3.5174	423.9388	4.4105	35562.3430
10000	8.1301	423.6748	12.0964	35561.4352
20000	16.7364	423.5561	23.7645	35561.2809
30000	16.9350	423.5149	22.8284	35560.4609
40000	22.3746	423.7382	31.8855	35560.6718
50000	28.7068	423.7666	38.8280	35560.9353
60000	33.6887	423.6112	46.3011	35560.9804

23–32% for most evolutionary cases, and 21–32% for pure Monte Carlo evaluation. These results confirm that hardware upgrades (higher core count, clock speeds) translate to tangible performance gains for compute-intensive iterative algorithms. The only outlier is map1 in the evolutionary pipeline, which is slower due to its dense graph structure requiring more computational effort per fitness evaluation. For reproducibility, we recommend documenting: (1) complete hardware specifications (CPU architecture, core count, memory); (2) software environment (Python version, library versions); (3) algorithm parameters including random seeds; and (4) runtime measurement methodology, as all these factors can significantly affect observed performance.

V Conclusion

This report presents a unified IEM pipeline with three core algorithms: Monte Carlo IC evaluation, MC-SAA greedy heuristic with IMRank screening, and pymoo-based binary evolutionary optimization. The formulation, pseudocode, and batch experiment protocol support reproducible evaluation across provided and generated datasets.

Experimental results demonstrate distinct scaling characteristics: the heuristic method achieves efficient runtimes (17–55s) on large graphs with up to 36742 nodes, with performance primarily governed by propagation probability and diffusion reach. The evolutionary method provides consistent performance (12–136s) across graphs with up to 13984 nodes. Cross-machine comparison shows the i7-14650HX achieves 1–8% speedup for heuristic, 23–32% for evolutionary (except map1), and 21–32% for Monte Carlo evaluation compared to the Ultra 7 258V.

Importantly, the two optimization methods were evaluated on different graph instances due to pipeline design requirements, so direct comparison of absolute scores or runtimes is not meaningful. Each method shows viable performance within its tested domain: the heuristic excels at handling large-scale sparse graphs through scenario sharing and incremental diffusion, while the evolutionary approach provides robust optimization across varying graph densities through population-based search and adaptive local refinement.

TABLE XII: Evaluator results (New Machine: i7-14650HX, 5000 simulations, Optimized)

Case	Time (s)	Score	Comparison vs Old
map1	2.6620	423.938800	1.32x faster (Old: 3.52s)
map2	3.6321	35562.343000	1.21x faster (Old: 4.41s)

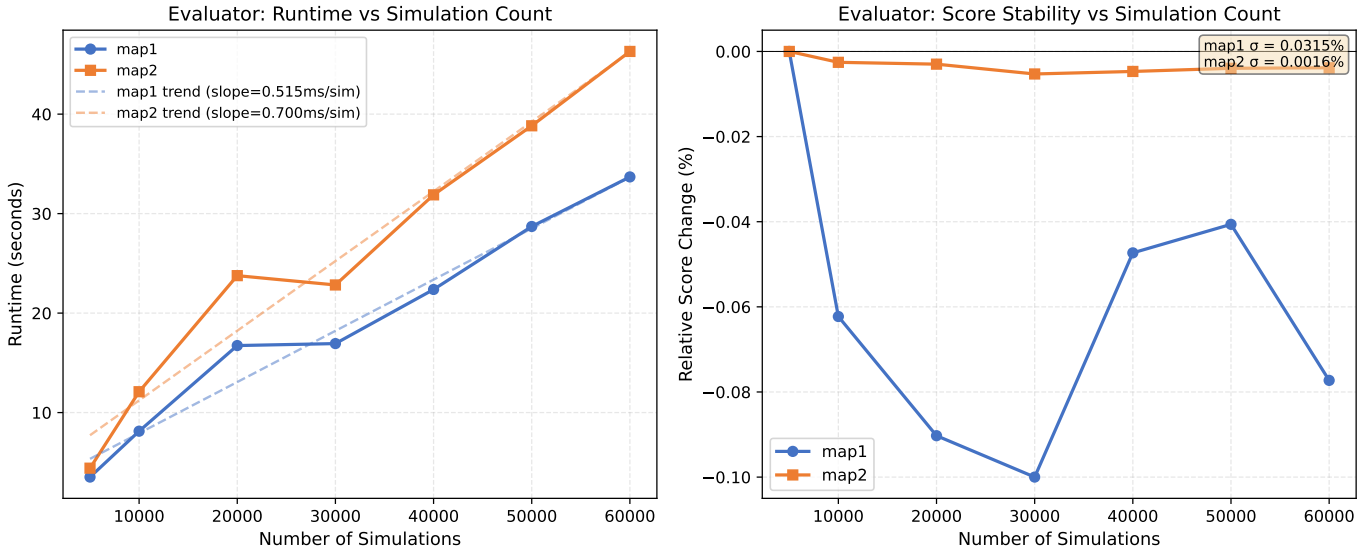

Fig. 4: Evaluator staged results: (Left) Runtime grows approximately linearly with simulation count for both map1 and map2; (Right) Score stability analysis showing relative changes from 5000-simulation baseline. Both cases exhibit high stability with standard deviations below 0.04%.

References

[1] K. Garimella, A. Gionis, N. Parotsidis, and N. Tatti, "Balancing information exposure in social networks," 2018. [Online]. Available: https://arxiv.org/abs/1709.01491

[2] S. Cheng, H.-W. Shen, J. Huang, W. Chen, and X.-Q. Cheng, "Imrank: Influence maximization via finding self-consistent ranking," 2014. [Online]. Available: https://arxiv.org/abs/1402.3939

[3] J. Wu, J. Gao, H. Zhu, and Z. Zhang, "Budgeted influence maximization via boost simulated annealing in social networks," 2022. [Online]. Available: https://arxiv.org/abs/2203.11594

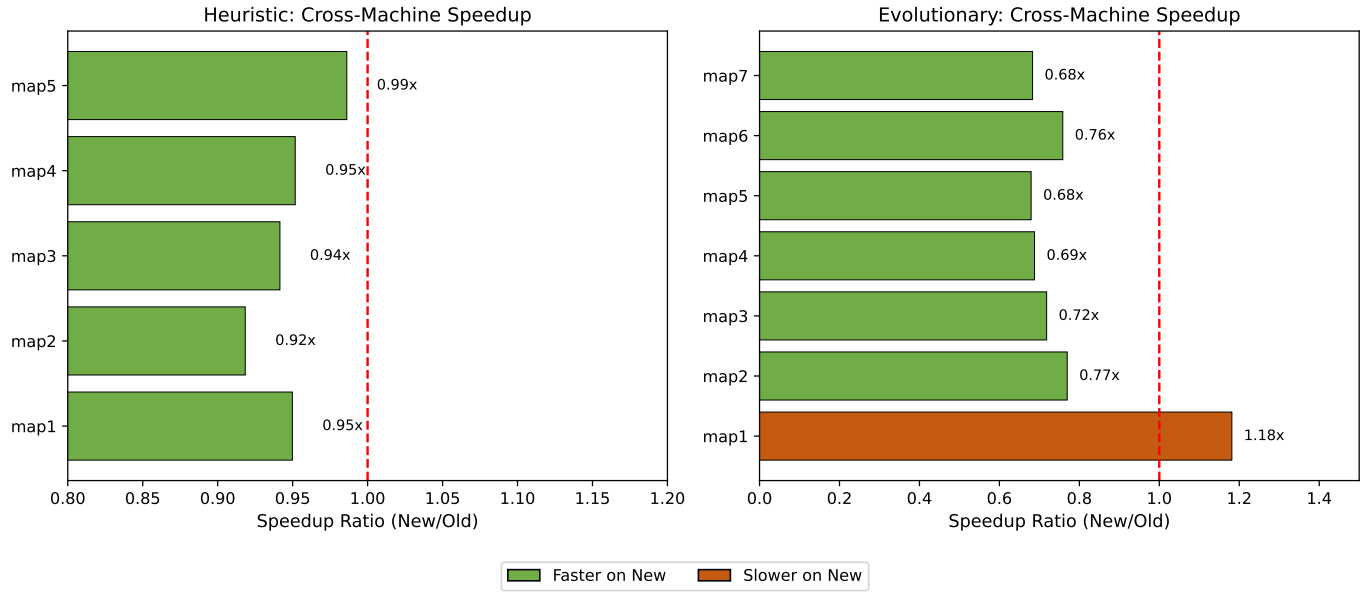

Fig. 5: Cross-machine speedup comparison: (Left) Heuristic method shows consistent but modest improvements (1–8% faster) on the new machine; (Right) Evolutionary method demonstrates significant speedups (23–32% faster) for map2–map7, with only map1 being slower (18%) on the new machine. Green bars indicate faster performance on the new machine, orange indicates slower.